SDK API Reference

Complete API reference for the Graph OLAP Python SDK - the **sole user interface** for the Graph OLAP Platform.

Overview

The Graph OLAP SDK provides the complete interface for all platform operations. There is no separate web console or GUI - all user interactions happen through this SDK in Jupyter notebooks.

Resource Managers Summary

Resource	Purpose	Role Required
`client.mappings`	Graph mapping CRUD (create, read, update, delete, copy, list)	Analyst
`client.instances`	Instance lifecycle (create, terminate, update CPU, connect)	Analyst
`client.snapshots`	Snapshot listing and status (creation is implicit)	Analyst
`client.favorites`	Bookmark mappings, snapshots, instances	Analyst
`client.schema`	Browse Starburst catalog metadata	Analyst
`client.health`	Basic health and readiness checks	None
`client.ops`	Cluster configuration and monitoring	Ops
`client.admin`	Bulk delete and privileged operations	Admin

Typical Workflow

```python
from graph_olap import GraphOLAPClient
from graph_olap_schemas import WrapperType

# 1. Connect to the platform
client = GraphOLAPClient.from_env()

# 2. Discover available data
catalogs = client.schema.list_catalogs()
tables = client.schema.search_tables("customer")

# 3. Create a mapping (defines graph structure)
mapping = client.mappings.create(
    name="Customer Network",
    node_definitions=[...],
    edge_definitions=[...],
)

# 4. Create and connect to an instance
instance = client.instances.create_from_mapping_and_wait(
    mapping_id=mapping.id,
    name="Analysis Instance",
    wrapper_type=WrapperType.RYUGRAPH,
)
conn = client.instances.connect(instance.id)

# 5. Query and analyze
df = conn.query_df("MATCH (n:Customer) RETURN n.name, n.city")
conn.algo.pagerank("Customer", "influence_score")

# 6. Clean up
client.instances.terminate(instance.id)
client.close()
```

---

1. GraphOLAPClient

The main entry point for interacting with the Graph OLAP Platform.

Class Definition

```python
class GraphOLAPClient:
    """
    Main client for the Graph OLAP Platform.
```

```
    Provides access to all platform resources through typed resource managers.
    Handles authentication, connection pooling, and retry logic.
    """
```

Constructor

```
def __init__(
    self,
    api_url: str,
    api_key: str | None = None,
    internal_api_key: str | None = None,
    username: str | None = None,
    role: str | None = None,
    *,
    timeout: float = 30.0,
    max_retries: int = 3,
) -> None:
    """
    Initialize the Graph OLAP client.

    Args:
        api_url: Base URL for the control plane API (e.g.,
"https://graph.example.com")
        api_key: API key for authentication (Bearer token)
        internal_api_key: Internal API key (X-Internal-Api-Key header)
        username: Username for user-scoped routes (X-Username header)
        role: User role for X-User-Role header (e.g., "analyst", "admin",
"ops")
        timeout: Default request timeout in seconds (default: 30.0)
        max_retries: Maximum retry attempts for failed requests (default: 3)

    Note:
        At least one of api_key, internal_api_key, or username must be provided
        for authentication. Use GraphOLAPClient.from_env() for environment-
based
        configuration.

    Example:
        >>> client = GraphOLAPClient(
        ...     api_url="https://graph.example.com",
        ...     api_key="your-api-key",
        ...     timeout=60.0,
        ... )
    """
```

Class Methods

from_env

```python
@classmethod
def from_env(cls) -> "GraphOLAPClient":
    """
    Create client from environment variables.

    Reads configuration from:
        - GRAPH_OLAP_API_URL: API base URL
        - GRAPH_OLAP_API_KEY: API key for authentication
        - GRAPH_OLAP_TIMEOUT: Request timeout (optional, default: 30.0)
        - GRAPH_OLAP_MAX_RETRIES: Max retries (optional, default: 3)

    Returns:
        Configured GraphOLAPClient instance

    Raises:
        EnvironmentError: If required variables are not set

    Example:
        >>> import os
        >>> os.environ["GRAPH_OLAP_API_URL"] = "https://graph.example.com"
        >>> os.environ["GRAPH_OLAP_API_KEY"] = "your-api-key"
        >>> client = GraphOLAPClient.from_env()
    """
```

Resource Managers

The client exposes typed resource managers as attributes:

```python
# Resource managers (read-only properties)
mappings: MappingResource      # Manage graph mappings
snapshots: SnapshotResource    # Manage data snapshots
instances: InstanceResource    # Manage graph instances
favorites: FavoriteResource    # Manage user favorites
schema: SchemaResource         # Browse Starburst schema metadata
ops: OpsResource               # Operations configuration (Ops role required)
health: HealthResource         # Health and status checks
admin: AdminResource           # Admin operations (Admin role required)
```

Instance Methods

close

```python
def close(self) -> None:
    """
    Close the client and release resources.

    Should be called when done with the client, or use as context manager.

    Example:
        >>> client = GraphOLAPClient(api_url, api_key)
        >>> try:
        ...     # Use client
        ... finally:
        ...     client.close()
    """
```

Context Manager Support

```python
def __enter__(self) -> "GraphOLAPClient":
    """Enter context manager."""
    return self

def __exit__(self, *args) -> None:
    """Exit context manager and close client."""
    self.close()
```

Example: Using as Context Manager

```python
with GraphOLAPClient(api_url, api_key) as client:
    mappings = client.mappings.list()
    # Client automatically closed on exit
```

---

2. MappingResource

Manage graph mapping definitions.

Methods

list

```python
def list(
    self,
    *,
    owner: str | None = None,
    search: str | None = None,
    created_after: str | None = None,
    created_before: str | None = None,
    sort_by: str = "created_at",
    sort_order: str = "desc",
    offset: int = 0,
    limit: int = 50,
) -> PaginatedList[Mapping]:
    """
    List mappings with optional filtering.

    Args:
        owner: Filter by owner username (Admin/Ops can see others' mappings)
        search: Search in name and description
        created_after: Filter by creation date (ISO 8601)
        created_before: Filter by creation date (ISO 8601)
        sort_by: Sort field (default: "created_at")
        sort_order: Sort order ("asc" or "desc", default: "desc")
        offset: Pagination offset (default: 0)
        limit: Maximum results (default: 50, max: 1000)

    Returns:
        PaginatedList[Mapping] with items, total, offset, limit attributes

    Example:
        >>> result = client.mappings.list()
        >>> for m in result.items:
        ...     print(f"{m.name}: {m.snapshot_count} snapshots")
        >>> print(f"Total: {result.total}")

        >>> # Search by name
        >>> result = client.mappings.list(search="customer")
    """
```

get

```python
def get(self, mapping_id: int) -> Mapping:
    """
    Get a single mapping by ID.

    Args:
        mapping_id: Mapping ID

    Returns:
        Mapping object with version details included
```

```
    Raises:
        NotFoundError: If mapping does not exist
        PermissionDeniedError: If user cannot access this mapping

    Example:
        >>> mapping = client.mappings.get(123)
        >>> print(mapping.version.node_definitions)
    """
```

create

```
def create(
    self,
    name: str,
    node_definitions: list[NodeDefinition],
    edge_definitions: list[EdgeDefinition],
    description: str | None = None,
    ttl: str | None = None,
    inactivity_timeout: str | None = None,
) -> Mapping:
    """
    Create a new mapping.

    Args:
        name: Display name for the mapping
        node_definitions: List of node table definitions
        edge_definitions: List of edge table definitions
        description: Optional description
        ttl: Time-to-live (ISO 8601 duration, e.g., "P30D" for 30 days)
        inactivity_timeout: Inactivity timeout (ISO 8601 duration)

    Returns:
        Created Mapping object

    Raises:
        ValidationError: If definitions are invalid
        ConflictError: If name already exists for this user

    Example:
        >>> mapping = client.mappings.create(
        ...     name="Customer Graph",
        ...     description="Customer purchase relationships",
        ...     node_definitions=[
        ...         NodeDefinition(
        ...             label="Customer",
        ...             sql="SELECT id, name FROM customers",
        ...             primary_key={"name": "id", "type": "STRING"},
        ...             properties=[{"name": "name", "type": "STRING"}],
        ...         ),
        ...     ],
```

```
        ...     edge_definitions=[
        ...         EdgeDefinition(
        ...             type="PURCHASED",
        ...             from_node="Customer",
        ...             to_node="Product",
        ...             sql="SELECT customer_id, product_id FROM orders",
        ...             from_key="customer_id",
        ...             to_key="product_id",
        ...             properties=[],
        ...         ),
        ...     ],
        ... )
    """
```

update

```
def update(
    self,
    mapping_id: int,
    change_description: str,
    *,
    name: str | None = None,
    description: str | None = None,
    node_definitions: list[NodeDefinition] | None = None,
    edge_definitions: list[EdgeDefinition] | None = None,
) -> Mapping:
    """
    Update a mapping (creates new version if definitions changed).

    Args:
        mapping_id: Mapping ID to update
        change_description: Description of changes (REQUIRED for audit trail)
        name: New name (optional)
        description: New description (optional)
        node_definitions: New node definitions (creates new version)
        edge_definitions: New edge definitions (creates new version)

    Returns:
        Updated Mapping object

    Raises:
        NotFoundError: If mapping does not exist
        ValidationError: If definitions are invalid

    Example:
        >>> # Update metadata only (no new version)
        >>> mapping = client.mappings.update(
        ...     123,
        ...     "Updated description text",
        ...     description="Updated description",
        ... )
```

```
        >>> # Update definitions (creates new version)
        >>> mapping = client.mappings.update(
        ...     123,
        ...     "Added age property to Customer",
        ...     node_definitions=[...],
        ... )
    """
```

delete

```
def delete(self, mapping_id: int) -> None:
    """
    Delete a mapping.

    Args:
        mapping_id: Mapping ID to delete

    Raises:
        NotFoundError: If mapping does not exist
        DependencyError: If mapping has snapshots

    Example:
        >>> client.mappings.delete(123)
    """
```

copy

```
def copy(self, mapping_id: int, new_name: str) -> Mapping:
    """
    Copy a mapping to a new mapping.

    Creates an exact copy with a new name. Useful for creating variations
    of existing mappings.

    Args:
        mapping_id: Source mapping ID
        new_name: Name for the new mapping

    Returns:
        New Mapping object (copies latest version with same definitions)

    Example:
        >>> new_mapping = client.mappings.copy(123, "Customer Graph - Q4
Version")
    """
```

get_version

```python
def get_version(self, mapping_id: int, version: int) -> MappingVersion:
    """
    Get a specific version of a mapping.

    Args:
        mapping_id: Mapping ID
        version: Version number

    Returns:
        MappingVersion object

    Raises:
        NotFoundError: If mapping or version does not exist

    Example:
        >>> v1 = client.mappings.get_version(123, version=1)
        >>> print(f"Created by: {v1.created_by_name}")
    """
```

list_versions

```python
def list_versions(
    self,
    mapping_id: int,
    limit: int = 50,
    offset: int = 0,
) -> list[MappingVersion]:
    """
    List all versions of a mapping.

    Args:
        mapping_id: Mapping ID
        limit: Maximum versions to return
        offset: Pagination offset

    Returns:
        List of MappingVersion objects (newest first)

    Example:
        >>> versions = client.mappings.list_versions(123)
        >>> for v in versions:
        ...     print(f"v{v.version}: {v.change_description}")
    """
```

diff_versions

```python
def diff_versions(
    self,
    mapping_id: int,
    from_version: int,
    to_version: int,
) -> dict:
    """
    Compare two mapping versions.

    Args:
        mapping_id: Mapping ID
        from_version: Base version number
        to_version: Target version number

    Returns:
        Diff dict with:
            - summary: {nodes_added, nodes_removed, edges_added, ...}
            - nodes: {added: [...], removed: [...], modified: [...]}
            - edges: {added: [...], removed: [...], modified: [...]}

    Example:
        >>> diff = client.mappings.diff_versions(123, from_version=1,
to_version=2)
        >>> print(f"Added {diff['summary']['nodes_added']} node definitions")
        >>> for node in diff['nodes']['added']:
        ...     print(f"  + {node['label']}")
    """
```

list_snapshots

```python
def list_snapshots(
    self,
    mapping_id: int,
    *,
    offset: int = 0,
    limit: int = 50,
) -> PaginatedList[Snapshot]:
    """
    List all snapshots for a mapping.

    Args:
        mapping_id: Mapping ID
        offset: Pagination offset (default: 0)
        limit: Maximum results (default: 50)

    Returns:
        PaginatedList[Snapshot] with items, total, offset, limit attributes

    Example:
```

```
        >>> result = client.mappings.list_snapshots(123)
        >>> for s in result.items:
        ...     print(f"{s.name}: {s.status}")
    """
```

list_instances

```
def list_instances(
    self,
    mapping_id: int,
    *,
    offset: int = 0,
    limit: int = 50,
) -> PaginatedList[Instance]:
    """
    List instances created from any snapshot of this mapping.

    Returns instances across all versions of the mapping, ordered by
    creation date (newest first). Use this to find all active graph
    instances associated with a mapping.

    Args:
        mapping_id: Mapping ID
        offset: Number of records to skip (default: 0)
        limit: Max records to return (default: 50, max: 100)

    Returns:
        PaginatedList[Instance] with items, total, offset, limit attributes

    Example:
        >>> # Find all instances for a mapping
        >>> instances = client.mappings.list_instances(mapping_id=123)
        >>> for i in instances:
        ...     print(f"{i.name}: {i.status}")

        >>> # Check if any instances are running
        >>> running = [i for i in instances if i.status == "running"]
        >>> print(f"{len(running)} instances currently running")
    """
```

set_lifecycle

```
def set_lifecycle(
    self,
    mapping_id: int,
    ttl: str | None = None,
    inactivity_timeout: str | None = None,
```

```
) -> Mapping:
    """
    Set lifecycle parameters for a mapping.

    Args:
        mapping_id: Mapping ID
        ttl: Time-to-live (ISO 8601 duration) or None to clear
        inactivity_timeout: Inactivity timeout (ISO 8601 duration) or None to
clear

    Returns:
        Updated Mapping object

    Example:
        >>> # Set 90-day TTL
        >>> mapping = client.mappings.set_lifecycle(123, ttl="P90D")

        >>> # Set 30-day inactivity timeout
        >>> mapping = client.mappings.set_lifecycle(
        ...     123,
        ...     inactivity_timeout="P30D",
        ... )
    """
```

get_tree

```
def get_tree(
    self,
    mapping_id: int,
    include_instances: bool = True,
    status: str | None = None,
) -> dict:
    """
    Get full resource hierarchy for a mapping.

    Returns the complete tree: versions -> snapshots -> instances.
    Useful for understanding dependencies before deletion.

    Args:
        mapping_id: Mapping ID
        include_instances: Include instance details (default: True)
        status: Filter snapshots by status (pending, creating, ready, failed)

    Returns:
        Tree structure dict

    Example:
        >>> tree = client.mappings.get_tree(123)
        >>> for version in tree['versions']:
        ...     print(f"v{version['version']}: {len(version['snapshots'])}
```

```
snapshots")
    """
```

---

3. SnapshotResource

> **SNAPSHOT FUNCTIONALITY DISABLED**
>
> Explicit snapshot APIs have been disabled. Instances are now created directly from mappings without requiring explicit snapshot creation. The snapshot layer operates implicitly when instances are created.
>
> Use `client.instances.create_from_mapping()` or `client.instances.create_from_mapping_and_wait()` instead of the snapshot methods described below.

Manage data snapshots exported from mappings.

Methods

list

```
def list(
    self,
    *,
    mapping_id: int | None = None,
    mapping_version: int | None = None,
    owner: str | None = None,
    status: str | None = None,
    search: str | None = None,
    created_after: str | None = None,
    created_before: str | None = None,
    sort_by: str = "created_at",
    sort_order: str = "desc",
    offset: int = 0,
    limit: int = 50,
) -> PaginatedList[Snapshot]:
    """
    List snapshots with optional filtering.

    Args:
        mapping_id: Filter by mapping ID
        mapping_version: Filter by mapping version
        owner: Filter by owner username
```

```
        status: Filter by status (pending, creating, ready, failed)
        search: Search in name and description
        created_after: Filter by creation date (ISO 8601)
        created_before: Filter by creation date (ISO 8601)
        sort_by: Sort field (default: "created_at")
        sort_order: Sort order ("asc" or "desc", default: "desc")
        offset: Pagination offset (default: 0)
        limit: Maximum results (default: 50, max: 1000)

    Returns:
        PaginatedList[Snapshot] with items, total, offset, limit attributes

    Example:
        >>> result = client.snapshots.list(mapping_id=123)
        >>> for s in result.items:
        ...     print(f"{s.name}: {s.status}")

        >>> result = client.snapshots.list(status="ready")
    """
```

get

```
def get(self, snapshot_id: int) -> Snapshot:
    """
    Get a single snapshot by ID.

    Args:
        snapshot_id: Snapshot ID

    Returns:
        Snapshot object

    Raises:
        NotFoundError: If snapshot does not exist

    Example:
        >>> snapshot = client.snapshots.get(456)
        >>> print(f"Status: {snapshot.status}, Size: {snapshot.size_bytes}")
    """
```

create

```
def create(
    self,
    mapping_id: int,
    name: str,
    *,
```

```
    mapping_version: int | None = None,
    description: str | None = None,
    ttl: str | None = None,
    inactivity_timeout: str | None = None,
) -> Snapshot:
    """
    Create a new snapshot (triggers data export).

    Args:
        mapping_id: Source mapping ID
        name: Display name
        mapping_version: Mapping version to use (defaults to current version)
        description: Optional description
        ttl: Time-to-live (ISO 8601 duration)
        inactivity_timeout: Inactivity timeout (ISO 8601 duration)

    Returns:
        Snapshot object with status='pending'

    Raises:
        NotFoundError: If mapping does not exist
        ValidationError: If mapping_version does not exist

    Example:
        >>> snapshot = client.snapshots.create(
        ...     mapping_id=123,
        ...     name="Q4 2024 Data",
        ...     description="End of quarter snapshot",
        ... )
        >>> print(f"Export started: {snapshot.id}")
    """
```

delete

```
def delete(self, snapshot_id: int) -> None:
    """
    Delete a snapshot.

    Args:
        snapshot_id: Snapshot ID to delete

    Raises:
        NotFoundError: If snapshot does not exist
        DependencyError: If snapshot has active instances

    Example:
        >>> client.snapshots.delete(456)
    """
```

wait_until_ready

```
def wait_until_ready(
    self,
    snapshot_id: int,
    timeout: int = 600,
    poll_interval: int = 5,
) -> Snapshot:
    """
    Wait for a snapshot to become ready.

    Polls the snapshot status until it reaches 'ready' or 'failed'.

    Args:
        snapshot_id: Snapshot ID
        timeout: Maximum wait time in seconds (default: 600)
        poll_interval: Time between status checks (default: 5)

    Returns:
        Snapshot object with status='ready'

    Raises:
        SnapshotFailedError: If snapshot export failed
        SDKTimeoutError: If timeout exceeded

    Example:
        >>> snapshot = client.snapshots.create(mapping_id=123, name="Test")
        >>> snapshot = client.snapshots.wait_until_ready(snapshot.id,
timeout=300)
        >>> print(f"Ready! {snapshot.node_counts}")
    """
```

create_and_wait

```
def create_and_wait(
    self,
    mapping_id: int,
    name: str,
    *,
    mapping_version: int | None = None,
    description: str | None = None,
    ttl: str | None = None,
    inactivity_timeout: str | None = None,
    timeout: int = 600,
    poll_interval: int = 5,
    on_progress: Callable[[str, int, int], None] | None = None,
) -> Snapshot:
    """
    Create a snapshot and wait for it to become ready.
```

```
    Convenience method combining create() and wait_until_ready().

    Args:
        mapping_id: Source mapping ID
        name: Display name
        mapping_version: Mapping version (defaults to current)
        description: Optional description
        ttl: Time-to-live (ISO 8601 duration)
        inactivity_timeout: Inactivity timeout (ISO 8601 duration)
        timeout: Maximum wait time in seconds
        poll_interval: Time between status checks
        on_progress: Optional callback(phase, completed_steps, total_steps)

    Returns:
        Snapshot object with status='ready'

    Example:
        >>> def progress(phase, done, total):
        ...     print(f"{phase}: {done}/{total}")
        >>> snapshot = client.snapshots.create_and_wait(
        ...     mapping_id=123,
        ...     name="Quick Snapshot",
        ...     on_progress=progress,
        ... )
    """
```

get_progress

```
def get_progress(self, snapshot_id: int) -> SnapshotProgress:
    """
    Get detailed export progress for a snapshot.

    Args:
        snapshot_id: Snapshot ID

    Returns:
        SnapshotProgress object with phase, steps, and completion info

    Example:
        >>> progress = client.snapshots.get_progress(456)
        >>> print(f"Phase: {progress.phase}")
        >>> print(f"Progress:
{progress.completed_steps}/{progress.total_steps}")
    """
```

update

```python
def update(
    self,
    snapshot_id: int,
    *,
    name: str | None = None,
    description: str | None = None,
) -> Snapshot:
    """
    Update snapshot metadata.

    Args:
        snapshot_id: Snapshot ID
        name: New name (optional)
        description: New description (optional)

    Returns:
        Updated Snapshot object

    Example:
        >>> snapshot = client.snapshots.update(456, name="Q4 Data Snapshot")
    """
```

retry

```python
def retry(self, snapshot_id: int) -> Snapshot:
    """
    Retry a failed snapshot export.

    Args:
        snapshot_id: ID of the failed snapshot

    Returns:
        Snapshot object with status='pending'

    Raises:
        InvalidStateError: If snapshot is not in 'failed' status

    Example:
        >>> if snapshot.status == "failed":
        ...     snapshot = client.snapshots.retry(snapshot.id)
        ...     snapshot = client.snapshots.wait_until_ready(snapshot.id)
    """
```

set_lifecycle

```python
def set_lifecycle(
    self,
    snapshot_id: int,
    ttl: str | None = None,
    inactivity_timeout: str | None = None,
) -> Snapshot:
    """
    Set lifecycle parameters for a snapshot.

    Args:
        snapshot_id: Snapshot ID
        ttl: Time-to-live (ISO 8601 duration) or None to clear
        inactivity_timeout: Inactivity timeout (ISO 8601 duration) or None to
clear

    Returns:
        Updated Snapshot object

    Example:
        >>> snapshot = client.snapshots.set_lifecycle(456, ttl="P14D")
    """
```

---

4. InstanceResource

Manage running graph instances.

Methods

list

```python
def list(
    self,
    *,
    snapshot_id: int | None = None,
    owner: str | None = None,
    status: str | None = None,
    search: str | None = None,
    created_after: str | None = None,
    created_before: str | None = None,
    sort_by: str = "created_at",
    sort_order: str = "desc",
    offset: int = 0,
    limit: int = 50,
) -> PaginatedList[Instance]:
```

```
    """
    List instances with optional filtering.

    Args:
        snapshot_id: Filter by snapshot ID
        owner: Filter by owner username
        status: Filter by status (starting, running, stopping, failed)
        search: Search in name and description
        created_after: Filter by creation date (ISO 8601)
        created_before: Filter by creation date (ISO 8601)
        sort_by: Sort field (default: "created_at")
        sort_order: Sort order ("asc" or "desc", default: "desc")
        offset: Pagination offset (default: 0)
        limit: Maximum results (default: 50, max: 1000)

    Returns:
        PaginatedList[Instance] with items, total, offset, limit attributes

    Example:
        >>> result = client.instances.list(status="running")
        >>> for i in result.items:
        ...     print(f"{i.name}: {i.instance_url}")
    """
```

get

```
def get(self, instance_id: int) -> Instance:
    """
    Get a single instance by ID.

    Args:
        instance_id: Instance ID

    Returns:
        Instance object

    Raises:
        NotFoundError: If instance does not exist

    Example:
        >>> instance = client.instances.get(789)
        >>> print(f"Status: {instance.status}, URL: {instance.instance_url}")
    """
```

create

> **DEPRECATED:** This method requires an explicit `snapshot_id`. Use `create_from_mapping()` or `create_from_mapping_and_wait()` instead, which

> manage snapshots automatically.

```python
def create(
    self,
    snapshot_id: int,
    name: str,
    wrapper_type: WrapperType,
    *,
    description: str | None = None,
    ttl: int | str | None = None,
    inactivity_timeout: int | str | None = None,
) -> Instance:
    """
    [DEPRECATED] Create a new graph instance from an explicit snapshot.

    .. deprecated::
        Use `create_from_mapping()` or `create_from_mapping_and_wait()`
instead.
        These methods manage snapshots automatically.

    Args:
        snapshot_id: Source snapshot ID (must be 'ready')
        name: Display name
        wrapper_type: Graph database wrapper type (REQUIRED)
            - WrapperType.RYUGRAPH: High-performance embedded graph
(recommended)
            - WrapperType.FALKORDB: In-memory graph for smaller datasets
        description: Optional description
        ttl: Time-to-live (hours as int, or ISO 8601 duration like "PT24H")
        inactivity_timeout: Inactivity timeout (hours as int, or ISO 8601
duration)

    Returns:
        Instance object with status='starting'

    Raises:
        NotFoundError: If snapshot does not exist
        InvalidStateError: If snapshot is not 'ready'
        ConcurrencyLimitError: If instance limits exceeded

    Example:
        >>> # DEPRECATED - Use create_from_mapping_and_wait() instead:
        >>> from graph_olap_schemas import WrapperType
        >>> instance = client.instances.create_from_mapping_and_wait(
        ...     mapping_id=123,
        ...     name="Analysis Instance",
        ...     wrapper_type=WrapperType.RYUGRAPH,
        ...     ttl=24,
        ... )
    """
```

update

> **Internal Use Only:** This method is primarily for internal metadata updates. Most users should set the name and description at instance creation time.

```
def update(
    self,
    instance_id: int,
    *,
    name: str | None = None,
    description: str | None = None,
) -> Instance:
    """
    Update instance metadata.

    Note: This is primarily for internal use. Set name and description
    at creation time via `create_from_mapping_and_wait()` instead.

    Args:
        instance_id: Instance ID
        name: New name (optional)
        description: New description (optional)

    Returns:
        Updated Instance object

    Example:
        >>> instance = client.instances.update(789, name="Renamed Instance")
    """
```

terminate

```
def terminate(self, instance_id: int) -> None:
    """
    Terminate and delete an instance.

    Immediately deletes the Kubernetes pod and removes the instance from the
    database. Use this to clean up instances when done.

    Args:
        instance_id: Instance ID to terminate

    Raises:
        NotFoundError: If instance does not exist

    Example:
        >>> client.instances.terminate(789)
```

```
        >>> # Instance is immediately deleted
    """
```

wait_until_running

> **DEPRECATED:** Use `create_from_mapping_and_wait()` instead, which handles the entire workflow (snapshot export + instance startup) automatically.

```
def wait_until_running(
    self,
    instance_id: int,
    timeout: int = 300,
    poll_interval: int = 5,
) -> Instance:
    """
    [DEPRECATED] Wait for an instance to become running.

    .. deprecated::
        Use `create_from_mapping_and_wait()` instead, which handles
        the entire workflow automatically.

    Args:
        instance_id: Instance ID
        timeout: Maximum wait time in seconds (default: 300)
        poll_interval: Time between status checks (default: 5)

    Returns:
        Instance object with status='running'

    Raises:
        InstanceFailedError: If instance startup failed
        SDKTimeoutError: If timeout exceeded

    Example:
        >>> # DEPRECATED - Use create_from_mapping_and_wait() instead:
        >>> from graph_olap_schemas import WrapperType
        >>> instance = client.instances.create_from_mapping_and_wait(
        ...     mapping_id=123,
        ...     name="Test",
        ...     wrapper_type=WrapperType.RYUGRAPH,
        ... )
        >>> print(f"Ready at: {instance.instance_url}")
    """
```

create_and_wait

> **DEPRECATED:** This method requires an explicit `snapshot_id`. Use `create_from_mapping_and_wait()` instead, which manages snapshots automatically.

```
def create_and_wait(
    self,
    snapshot_id: int,
    name: str,
    wrapper_type: WrapperType,
    *,
    description: str | None = None,
    ttl: int | str | None = None,
    inactivity_timeout: int | str | None = None,
    timeout: int = 300,
    poll_interval: int = 5,
    on_progress: Callable[[str, int, int], None] | None = None,
) -> Instance:
    """
    [DEPRECATED] Create an instance and wait for it to become running.

    .. deprecated::
        Use `create_from_mapping_and_wait()` instead, which manages
        snapshots automatically.

    Args:
        snapshot_id: Source snapshot ID
        name: Display name
        wrapper_type: Graph database wrapper type (REQUIRED)
            - WrapperType.RYUGRAPH: High-performance embedded graph
(recommended)
            - WrapperType.FALKORDB: In-memory graph for smaller datasets
        description: Optional description
        ttl: Time-to-live (hours as int, or ISO 8601 duration like "PT24H")
        inactivity_timeout: Inactivity timeout (hours as int, or ISO 8601
duration)
        timeout: Maximum wait time in seconds (default: 300)
        poll_interval: Time between status checks (default: 5)
        on_progress: Optional callback(phase, completed_steps, total_steps)

    Returns:
        Instance object with status='running'

    Example:
        >>> # DEPRECATED - Use create_from_mapping_and_wait() instead:
        >>> from graph_olap_schemas import WrapperType
        >>> instance = client.instances.create_from_mapping_and_wait(
        ...     mapping_id=123,
        ...     name="Quick Analysis",
        ...     wrapper_type=WrapperType.FALKORDB,
        ...     ttl=24,
        ... )
```

```
        >>> conn = client.instances.connect(instance.id)
    """
```

create_from_mapping

```
def create_from_mapping(
    self,
    mapping_id: int,
    name: str,
    wrapper_type: WrapperType,
    *,
    mapping_version: int | None = None,
    description: str | None = None,
    ttl: int | str | None = None,
    inactivity_timeout: int | str | None = None,
) -> Instance:
    """
    Create a new graph instance directly from a mapping.

    This is a convenience method that creates a snapshot automatically
    and queues instance creation. The snapshot is managed internally -
    users don't need to interact with snapshots when using this method.

    The instance will initially have status='waiting_for_snapshot' until
    the snapshot export completes, then transition to 'starting'.

    Args:
        mapping_id: Source mapping ID
        name: Display name for the instance
        wrapper_type: Graph database wrapper type (FALKORDB or RYUGRAPH)
        mapping_version: Mapping version to use (defaults to current version)
        description: Optional description
        ttl: Time-to-live (hours as int, or ISO 8601 duration like "PT48H")
        inactivity_timeout: Inactivity timeout (hours or ISO 8601)

    Returns:
        Instance object with status='waiting_for_snapshot'

    Raises:
        NotFoundError: If mapping doesn't exist
        ConcurrencyLimitError: If instance limits exceeded

    Example:
        >>> from graph_olap_schemas import WrapperType
        >>> instance = client.instances.create_from_mapping(
        ...     mapping_id=123,
        ...     name="Quick Analysis",
        ...     wrapper_type=WrapperType.FALKORDB,
        ... )
        >>> # Instance starts with status='waiting_for_snapshot'
        >>> print(instance.status)
```

```
        'waiting_for_snapshot'
    """
```

create_from_mapping_and_wait

```
def create_from_mapping_and_wait(
    self,
    mapping_id: int,
    name: str,
    wrapper_type: WrapperType,
    *,
    mapping_version: int | None = None,
    description: str | None = None,
    ttl: int | str | None = None,
    inactivity_timeout: int | str | None = None,
    timeout: int = 900,
    poll_interval: int = 5,
    on_progress: Callable[[str, int, int], None] | None = None,
) -> Instance:
    """
    Create an instance from a mapping and wait for it to become running.

    This is the recommended method for most use cases. It handles the
    entire workflow automatically:

    1. Creates a snapshot from the mapping (managed internally)
    2. Waits for snapshot export to complete
    3. Creates the instance and waits for startup
    4. Returns when the instance is fully ready

    Args:
        mapping_id: Source mapping ID
        name: Display name for the instance
        wrapper_type: Graph database wrapper type (FALKORDB or RYUGRAPH)
        mapping_version: Mapping version to use (defaults to current)
        description: Optional description
        ttl: Time-to-live (hours as int, or ISO 8601 duration)
        inactivity_timeout: Inactivity timeout (hours or ISO 8601)
        timeout: Maximum wait time in seconds (default: 900 = 15 minutes)
        poll_interval: Time between status checks (default: 5)
        on_progress: Optional callback(phase, completed_steps, total_steps)

    Returns:
        Instance object with status='running'

    Raises:
        TimeoutError: If instance doesn't start within timeout
        InstanceFailedError: If instance or snapshot fails
        NotFoundError: If mapping doesn't exist
        ConcurrencyLimitError: If instance limits exceeded
```

```
    Example:
        >>> from graph_olap_schemas import WrapperType
        >>> instance = client.instances.create_from_mapping_and_wait(
        ...     mapping_id=123,
        ...     name="Quick Analysis",
        ...     wrapper_type=WrapperType.FALKORDB,
        ...     ttl=48,  # 48 hours
        ... )
        >>> # Instance is now running and ready
        >>> conn = client.instances.connect(instance.id)
        >>> result = conn.query("MATCH (n) RETURN count(n)")
    """
```

connect

```
def connect(self, instance_id: int) -> InstanceConnection:
    """
    Create a connection to a running instance.

    The connection provides query execution and algorithm access.

    Args:
        instance_id: Instance ID (must be 'running')

    Returns:
        InstanceConnection for querying the graph

    Raises:
        InvalidStateError: If instance is not 'running'

    Example:
        >>> conn = client.instances.connect(789)
        >>> result = conn.query("MATCH (n) RETURN count(n)")
        >>> conn.close()

        >>> # Or use as context manager
        >>> with client.instances.connect(789) as conn:
        ...     result = conn.query("MATCH (n) RETURN count(n)")
    """
```

get_progress

```
def get_progress(self, instance_id: int) -> InstanceProgress:
    """
    Get instance startup progress.

    Args:
```

```
        instance_id: Instance ID

    Returns:
        InstanceProgress object with phase, completed_steps, total_steps

    Example:
        >>> progress = client.instances.get_progress(789)
        >>> print(f"Phase: {progress.phase},
{progress.completed_steps}/{progress.total_steps}")
    """
```

get_health

```
def get_health(self, instance_id: int, *, timeout: float = 5.0) -> dict[str,
object]:
    """
    Get detailed health status of an instance.

    Args:
        instance_id: Instance ID
        timeout: Health check timeout in seconds (default: 5.0)

    Returns:
        Dict with health details including:
            - status: "healthy" | "unhealthy" | "unknown"
            - latency_ms: Response time
            - wrapper_version: Wrapper software version
            - database_status: Database connection status

    Example:
        >>> health = client.instances.get_health(789)
        >>> print(f"Status: {health['status']}, Latency:
{health['latency_ms']}ms")
    """
```

check_health

```
def check_health(self, instance_id: int, *, timeout: float = 5.0) -> bool:
    """
    Quick health check for an instance.

    Args:
        instance_id: Instance ID
        timeout: Health check timeout in seconds (default: 5.0)

    Returns:
        True if instance is healthy, False otherwise
```

```
    Example:
        >>> if client.instances.check_health(789):
        ...     print("Instance is healthy")
    """
```

extend_ttl

> **Prefer setting TTL at creation:** Set the `ttl` parameter when calling `create_from_mapping_and_wait()` to avoid needing runtime TTL extension. Use this method only when you need to extend an already-running instance.

```
def extend_ttl(
    self,
    instance_id: int,
    hours: int = 24,
) -> Instance:
    """
    Extend instance TTL by specified hours from current expiry.

    Note: Prefer setting `ttl` at creation time via
`create_from_mapping_and_wait()`
    to avoid needing runtime extension. Use this only for already-running
instances.

    Args:
        instance_id: Instance ID
        hours: Hours to add to current TTL (default: 24)

    Returns:
        Updated Instance object

    Raises:
        ValidationError: If extension exceeds maximum TTL (7 days from
creation)

    Example:
        >>> # Preferred: Set TTL at creation time
        >>> instance = client.instances.create_from_mapping_and_wait(
        ...     mapping_id=123, name="Analysis",
wrapper_type=WrapperType.RYUGRAPH,
        ...     ttl=48,  # Set 48-hour TTL upfront
        ... )

        >>> # Extension (for already-running instances only)
        >>> instance = client.instances.extend_ttl(789, hours=24)
    """
```

set_lifecycle

> **Prefer setting lifecycle at creation:** Set `ttl` and `inactivity_timeout` parameters when calling `create_from_mapping_and_wait()`. Use this method only when you need to modify an already-running instance.

```
def set_lifecycle(
    self,
    instance_id: int,
    ttl: str | None = None,
    inactivity_timeout: str | None = None,
) -> Instance:
    """
    Set lifecycle parameters for an instance.

    Note: Prefer setting `ttl` and `inactivity_timeout` at creation time via
    `create_from_mapping_and_wait()`. Use this only for already-running
instances.

    Args:
        instance_id: Instance ID
        ttl: Time-to-live (ISO 8601 duration) or None to clear
        inactivity_timeout: Inactivity timeout (ISO 8601 duration) or None to
clear

    Returns:
        Updated Instance object

    Example:
        >>> # Preferred: Set lifecycle at creation time
        >>> instance = client.instances.create_from_mapping_and_wait(
        ...     mapping_id=123, name="Analysis",
wrapper_type=WrapperType.RYUGRAPH,
        ...     ttl=48,               # 48 hours
        ...     inactivity_timeout=2,  # 2 hours
        ... )

        >>> # Runtime modification (for already-running instances only)
        >>> instance = client.instances.set_lifecycle(789, ttl="PT48H")
    """
```

update_cpu

```
def update_cpu(self, instance_id: int, cpu_cores: int) -> Instance:
    """
    Update CPU cores for a running instance.

    Uses Kubernetes in-place resize to change CPU without restarting.
```

```
    Args:
        instance_id: Instance ID
        cpu_cores: CPU cores (1-8). Sets request=N cores, limit=2N cores for
burst capacity.

    Returns:
        Updated Instance object

    Raises:
        InvalidStateError: If instance is not running
        ValidationError: If cpu_cores is out of range (1-8)

    Example:
        >>> instance = client.instances.update_cpu(789, cpu_cores=4)
        >>> print(f"Updated to {instance.cpu_cores} cores")
    """
```

---

5. InstanceConnection

Connection to a running graph instance for queries and algorithms.

Class Definition

```
class InstanceConnection:
    """
    Connection to a running graph instance.

    Provides methods for executing Cypher queries and graph algorithms.
    Supports both synchronous and context manager usage.
    """
```

Query Methods

query

```
def query(
    self,
    cypher: str,
    parameters: dict[str, Any] | None = None,
    *,
```

```
    timeout: float | None = None,
    coerce_types: bool = True,
) -> QueryResult:
    """
    Execute a Cypher query.

    Args:
        cypher: Cypher query string
        parameters: Query parameters (optional)
        timeout: Override default timeout in seconds (optional)
        coerce_types: Convert DATE/TIMESTAMP/INTERVAL to Python types (default:
True)

    Returns:
        QueryResult with columns, rows, and metadata

    Raises:
        QueryTimeoutError: If query exceeds timeout
        RyugraphError: If Cypher syntax is invalid

    Example:
        >>> result = conn.query(
        ...     "MATCH (c:Customer) WHERE c.city = $city RETURN c.name, c.age",
        ...     parameters={"city": "London"},
        ... )
        >>> for row in result:
        ...     print(row["c.name"], row["c.age"])
    """
```

query_df

```
def query_df(
    self,
    cypher: str,
    parameters: dict[str, Any] | None = None,
    *,
    backend: str = "polars",
) -> "pl.DataFrame | pd.DataFrame":
    """
    Execute a Cypher query and return as DataFrame directly.

    Args:
        cypher: Cypher query string
        parameters: Query parameters (optional)
        backend: DataFrame backend - "polars" (default) or "pandas"

    Returns:
        polars.DataFrame or pandas.DataFrame depending on backend

    Example:
        >>> # Polars (default)
```

```
        >>> df = conn.query_df("MATCH (n:Customer) RETURN n.name, n.age")
        >>> df.filter(pl.col("n.age") > 30)
        >>>
        >>> # Pandas
        >>> df = conn.query_df("MATCH (n:Customer) RETURN n.name",
backend="pandas")
    """
```

query_scalar

```
def query_scalar(
    self,
    cypher: str,
    parameters: dict[str, Any] | None = None,
) -> Any:
    """
    Execute a query expecting a single scalar value.

    Args:
        cypher: Cypher query returning a single value
        parameters: Query parameters (optional)

    Returns:
        The single scalar value (int, float, str, etc.)

    Raises:
        ValueError: If result has multiple rows or columns

    Example:
        >>> count = conn.query_scalar("MATCH (n) RETURN count(n)")
        >>> avg = conn.query_scalar("MATCH (n:Customer) RETURN avg(n.age)")
    """
```

query_one

```
def query_one(
    self,
    cypher: str,
    parameters: dict[str, Any] | None = None,
) -> dict[str, Any] | None:
    """
    Execute a query expecting a single row.

    Args:
        cypher: Cypher query returning single row
        parameters: Query parameters (optional)
```

```
    Returns:
        Dict of column->value for first row, or None if empty

    Example:
        >>> user = conn.query_one(
        ...     "MATCH (u:User {id: $id}) RETURN u.*",
        ...     {"id": 123},
        ... )
        >>> if user:
        ...     print(user["name"])
    """
```

Schema and Status Methods

get_schema

```
def get_schema(self) -> Schema:
    """
    Get the graph schema (node and relationship tables).

    Returns:
        Schema object with nodes and relationships definitions

    Example:
        >>> schema = conn.get_schema()
        >>> for label, info in schema.nodes.items():
        ...     print(f":{label} - {info['properties']}")
    """
```

get_lock

```
def get_lock(self) -> LockStatus:
    """
    Check if the instance is locked by an algorithm.

    Returns:
        LockStatus with lock information

    Example:
        >>> lock = conn.get_lock()
        >>> if lock.locked:
        ...     print(f"Locked by {lock.holder_name} for {lock.algorithm}")
    """
```

status

```python
def status(self) -> dict:
    """
    Get detailed instance status.

    Returns:
        Status dict with:
            - memory_usage_bytes: Current memory usage
            - disk_usage_bytes: Current disk usage
            - uptime_seconds: Instance uptime
            - graph_stats: {node_count, edge_count}
            - lock: Lock status information

    Example:
        >>> status = conn.status()
        >>> print(f"Memory: {status['memory_usage_bytes'] / 1024**3:.2f} GB")
        >>> print(f"Nodes: {status['graph_stats']['node_count']:,}")
    """
```

Algorithm Managers

The connection provides access to algorithm managers:

```python
# Native Ryugraph algorithms
algo: AlgorithmManager

# NetworkX algorithms
networkx: NetworkXManager
```

AlgorithmManager (conn.algo)

```python
def algorithms(self, category: str | None = None) -> list[dict]:
    """
    List available native Ryugraph algorithms.

    Args:
        category: Filter by category (centrality, community, pathfinding)

    Returns:
        List of algorithm info dicts

    Example:
        >>> algos = conn.algo.algorithms()
        >>> for algo in algos:
```

```
        ...     print(f"{algo['name']}: {algo['description']}")
    """

def algorithm_info(self, algorithm: str) -> dict:
    """
    Get detailed info for an algorithm.

    Args:
        algorithm: Algorithm name

    Returns:
        Dict with name, category, description, parameters

    Example:
        >>> info = conn.algo.algorithm_info("pagerank")
        >>> for param in info['parameters']:
        ...     print(f"  {param['name']}: {param['description']}")
    """

def run(
    self,
    algorithm: str,
    node_label: str | None = None,
    property_name: str | None = None,
    *,
    edge_type: str | None = None,
    params: dict | None = None,
    timeout: int = 300,
    wait: bool = True,
) -> AlgorithmExecution:
    """
    Run any native Ryugraph algorithm.

    Args:
        algorithm: Algorithm name (e.g., "pagerank", "wcc")
        node_label: Target node label
        property_name: Property to store results
        edge_type: Filter to specific edge type
        params: Algorithm-specific parameters
        timeout: Max wait time in seconds
        wait: Wait for completion (default: True)

    Returns:
        AlgorithmExecution with status and results

    Example:
        >>> exec = conn.algo.run(
        ...     "pagerank",
        ...     node_label="Customer",
        ...     property_name="pr_score",
        ...     params={"damping_factor": 0.85},
        ... )
    """
```

```python
def pagerank(
    self,
    node_label: str,
    property_name: str,
    damping: float = 0.85,
    iterations: int = 20,
    wait: bool = True,
    timeout: int = 300,
) -> AlgorithmExecution:
    """Run PageRank algorithm."""

def wcc(
    self,
    node_label: str,
    property_name: str,
    wait: bool = True,
    timeout: int = 300,
) -> AlgorithmExecution:
    """Run Weakly Connected Components."""

def louvain(
    self,
    node_label: str,
    property_name: str,
    wait: bool = True,
    timeout: int = 300,
) -> AlgorithmExecution:
    """Run Louvain community detection."""

def scc(
    self,
    node_label: str,
    property_name: str,
    wait: bool = True,
    timeout: int = 300,
) -> AlgorithmExecution:
    """Run Strongly Connected Components (Tarjan's)."""

def kcore(
    self,
    node_label: str,
    property_name: str,
    k: int = 1,
    wait: bool = True,
    timeout: int = 300,
) -> AlgorithmExecution:
    """Run k-core decomposition."""

def label_propagation(
    self,
    node_label: str,
    property_name: str,
    max_iterations: int = 100,
    wait: bool = True,
```

```
    timeout: int = 300,
) -> AlgorithmExecution:
    """Run label propagation community detection."""

def triangle_count(
    self,
    node_label: str,
    property_name: str,
    wait: bool = True,
    timeout: int = 300,
) -> AlgorithmExecution:
    """Count triangles for each node."""

def shortest_path(
    self,
    source_id: str,
    target_id: str,
    weight_property: str | None = None,
) -> dict:
    """
    Find shortest path between two nodes (synchronous).

    Args:
        source_id: Source node ID
        target_id: Target node ID
        weight_property: Edge property for weighted path

    Returns:
        Dict with path, length, and total_weight (if weighted)
    """
```

NetworkXManager (conn.networkx)

```
def algorithms(
    self,
    category: str | None = None,
    search: str | None = None,
) -> list[AlgorithmInfo]:
    """
    List available NetworkX algorithms.

    Args:
        category: Filter by category
        search: Search algorithm names

    Returns:
        List of AlgorithmInfo objects

    Example:
        >>> algos = conn.networkx.algorithms(category="centrality")
        >>> algos = conn.networkx.algorithms(search="community")
```

```python
    """

def algorithm_info(self, name: str) -> AlgorithmInfo:
    """
    Get detailed information about a NetworkX algorithm.

    Args:
        name: Algorithm name

    Returns:
        AlgorithmInfo with parameters and documentation
    """

def categories(self) -> list[str]:
    """Get available algorithm categories."""

def run(
    self,
    algorithm: str,
    *,
    node_label: str | None = None,
    property_name: str | None = None,
    params: dict | None = None,
    edge_types: list[str] | None = None,
    directed: bool = False,
    weight_property: str | None = None,
    source: str | None = None,
    target: str | None = None,
    subgraph_query: str | None = None,
    wait: bool = True,
    timeout: int = 300,
) -> AlgorithmExecution:
    """
    Run any NetworkX algorithm by name.

    Args:
        algorithm: NetworkX algorithm name
        node_label: Filter to specific node type
        property_name: Property to store results (omit to return directly)
        params: Algorithm-specific parameters
        edge_types: Filter to specific edge types
        directed: Treat graph as directed
        weight_property: Edge property for weights
        source: Source node ID (for path algorithms)
        target: Target node ID (for path algorithms)
        subgraph_query: Cypher query to select subset
        wait: Wait for completion
        timeout: Max wait time in seconds

    Returns:
        AlgorithmExecution with results

    Example:
        >>> result = conn.networkx.run(
```

```
        ...     "betweenness_centrality",
        ...     node_label="Customer",
        ...     property_name="betweenness",
        ...     params={"k": 100},
        ... )
    """

def degree_centrality(
    self,
    node_label: str,
    property_name: str,
    wait: bool = True,
    timeout: int = 300,
) -> AlgorithmExecution:
    """Run degree centrality."""

def betweenness_centrality(
    self,
    node_label: str,
    property_name: str,
    normalized: bool = True,
    k: int | None = None,
    wait: bool = True,
    timeout: int = 300,
) -> AlgorithmExecution:
    """Run betweenness centrality."""

def closeness_centrality(
    self,
    node_label: str,
    property_name: str,
    normalized: bool = True,
    wait: bool = True,
    timeout: int = 300,
) -> AlgorithmExecution:
    """Run closeness centrality."""

def eigenvector_centrality(
    self,
    node_label: str,
    property_name: str,
    max_iter: int = 100,
    tol: float = 1e-6,
    wait: bool = True,
    timeout: int = 300,
) -> AlgorithmExecution:
    """Run eigenvector centrality."""

def clustering_coefficient(
    self,
    node_label: str,
    property_name: str,
    wait: bool = True,
    timeout: int = 300,
```

```
) -> AlgorithmExecution:
    """Run clustering coefficient."""
```

Connection Lifecycle

```
def close(self) -> None:
    """Close the connection and release resources."""

def __enter__(self) -> "InstanceConnection":
    """Enter context manager."""

def __exit__(self, *args) -> None:
    """Exit context manager and close connection."""
```

---

6. QueryResult

Result of a Cypher query with multiple output format options.

Class Definition

```
@dataclass
class QueryResult:
    """
    Result of a Cypher query.

    Analysts can convert results to their preferred format:
    - DataFrames (polars/pandas) for tabular analysis
    - Dicts for programmatic access
    - NetworkX for graph algorithms
    - Scalar for single values

    Attributes:
        columns: Column names
        column_types: Ryugraph types (STRING, INT64, DATE, etc.)
        rows: List of row values
        row_count: Number of rows
        execution_time_ms: Query execution time
    """
    columns: list[str]
    column_types: list[str]
    rows: list[list]
```

```
    row_count: int
    execution_time_ms: int
```

DataFrame Conversion

to_polars

```
def to_polars(self) -> "pl.DataFrame":
    """
    Convert to Polars DataFrame.

    Returns:
        polars.DataFrame with typed columns

    Example:
        >>> df = result.to_polars()
        >>> df.filter(pl.col("age") > 30).head()
    """
```

to_pandas

```
def to_pandas(self) -> "pd.DataFrame":
    """
    Convert to Pandas DataFrame.

    Returns:
        pandas.DataFrame

    Example:
        >>> df = result.to_pandas()
        >>> df[df["age"] > 30].head()
    """
```

Dict/JSON Conversion

to_dicts

```
def to_dicts(self) -> list[dict]:
    """
    Convert to list of dictionaries.

    Returns:
```

```
        List of dicts, one per row

    Example:
        >>> rows = result.to_dicts()
        >>> rows[0]  # {'name': 'Alice', 'age': 30}
    """
```

to_json

```
def to_json(self, path: str | None = None, indent: int = 2) -> str | None:
    """
    Convert to JSON string or write to file.

    Args:
        path: Optional file path to write to
        indent: JSON indentation (default: 2)

    Returns:
        JSON string if path is None, else None
    """
```

Scalar Extraction

scalar

```
def scalar(self) -> Any:
    """
    Extract single scalar value.

    Use for queries that return a single value like COUNT(*), SUM(), etc.

    Returns:
        The single value (or None if empty)

    Raises:
        ValueError: If result has multiple rows or columns

    Example:
        >>> count = result.scalar()
    """
```

first

```python
def first(self) -> dict | None:
    """
    Get first row as dict, or None if empty.

    Example:
        >>> user = result.first()
        >>> if user:
        ...     print(user["name"])
    """
```

Graph Conversion

to_networkx

```python
def to_networkx(self, directed: bool = True) -> "nx.Graph":
    """
    Convert to NetworkX graph.

    Works when query returns nodes and relationships.
    Nodes are identified by _id, properties become attributes.

    Args:
        directed: If True, return DiGraph; else Graph

    Returns:
        NetworkX Graph or DiGraph

    Example:
        >>> result = conn.query("MATCH (a)-[r]->(b) RETURN a, r, b")
        >>> G = result.to_networkx()
        >>> nx.pagerank(G)
    """
```

Export Methods

to_csv

```python
def to_csv(self, path: str, **kwargs) -> None:
    """
    Export to CSV file.

    Args:
        path: Output file path
        **kwargs: Passed to polars write_csv()
```

```
    Example:
        >>> result.to_csv("output.csv")
    """
```

to_parquet

```
def to_parquet(self, path: str, **kwargs) -> None:
    """
    Export to Parquet file.

    Args:
        path: Output file path
        **kwargs: Passed to polars write_parquet()

    Example:
        >>> result.to_parquet("output.parquet", compression="snappy")
    """
```

Visualization Methods

show

```
def show(
    self,
    as_type: str | None = None,
    **kwargs,
) -> Any:
    """
    Smart visualization - automatically picks the best display format.

    Auto-detects if data contains graphs or tabular data and chooses
    appropriate visualization based on data size.

    Args:
        as_type: Force visualization type ("table", "graph", "chart", "json")
        **kwargs: Passed to underlying visualization

    Size-Based Auto-Selection:
        - Tabular data, <=100 rows: Simple HTML table
        - Tabular data, >100 rows: Interactive DataTable (itables)
        - Graph data, <=5K nodes: PyVis interactive graph
        - Graph data, 5K-50K nodes: ipycytoscape
        - Graph data, >50K nodes: Graphistry

    Example:
```

```
        >>> result.show()  # Auto-detect best visualization
        >>> result.show("table")  # Force table view
        >>> result.show("graph", layout="dagre")
    """
```

to_itables

```
def to_itables(self, **kwargs) -> None:
    """
    Display as interactive DataTable using itables.

    Provides sorting, filtering, search, and pagination.

    Args:
        **kwargs: Passed to itables.show()

    Example:
        >>> result.to_itables(lengthMenu=[25, 50, 100])
    """
```

to_pyvis

```
def to_pyvis(
    self,
    height: str = "600px",
    notebook: bool = True,
    **kwargs,
) -> "Network":
    """
    Display as interactive PyVis network.

    Args:
        height: Visualization height
        notebook: Configure for Jupyter
        **kwargs: Additional PyVis options

    Returns:
        pyvis.network.Network

    Example:
        >>> net = result.to_pyvis()
        >>> net.show("graph.html")
    """
```

to_cytoscape

```python
def to_cytoscape(
    self,
    layout: str = "cose",
    directed: bool = True,
    node_color_property: str | None = None,
    node_size_property: str | None = None,
    **kwargs,
) -> "CytoscapeWidget":
    """
    Display as ipycytoscape widget.

    Args:
        layout: Layout algorithm ("cose", "dagre", "klay", etc.)
        directed: Show directed edges
        node_color_property: Property for color mapping
        node_size_property: Property for size mapping

    Returns:
        ipycytoscape.CytoscapeWidget
    """
```

Iteration Support

```python
def __iter__(self) -> Iterator[dict]:
    """Iterate over rows as dicts."""

def __len__(self) -> int:
    """Return row count."""

def __getitem__(self, index: int) -> dict:
    """Get row by index as dict."""

def __bool__(self) -> bool:
    """True if result has rows."""
```

---

7. Common Types

WrapperType

```python
from graph_olap_schemas import WrapperType
```

```python
class WrapperType(str, Enum):
    """
    Supported graph database wrapper types.

    Import from graph_olap_schemas package (not graph_olap).
    """
    RYUGRAPH = "ryugraph"   # High-performance embedded graph database
(recommended)
    FALKORDB = "falkordb"   # In-memory graph for smaller datasets
```

Usage:

```python
from graph_olap_schemas import WrapperType

# Use enum value for instance creation (recommended)
instance = client.instances.create_from_mapping_and_wait(
    mapping_id=123,
    name="My Instance",
    wrapper_type=WrapperType.RYUGRAPH,
)

# String values also accepted (but enum preferred for type safety)
instance = client.instances.create_from_mapping_and_wait(
    mapping_id=123,
    name="My Instance",
    wrapper_type="falkordb",  # Also works
)
```

Wrapper Comparison:

Feature	RYUGRAPH	FALKORDB
Storage	Disk-backed	In-memory
Max Graph Size	Large (100M+ nodes)	Medium (~10M nodes)
Query Performance	Excellent	Very fast (memory)
Startup Time	Slower (data loading)	Fast
Use Case	Production analytics	Quick exploration

PaginatedList

```python
@dataclass
class PaginatedList[T]:
    """
    Paginated list result from list() methods.

    All list() methods return this type for consistent pagination.
    """
    items: list[T]    # List of items for current page
    total: int        # Total number of items across all pages
    offset: int       # Current offset
    limit: int        # Page size

    def __iter__(self):
        """Iterate over items in current page."""
        return iter(self.items)

    def __len__(self):
        """Number of items in current page."""
        return len(self.items)
```

Usage:

```python
# List returns PaginatedList, not list
result = client.mappings.list(limit=10)

# Access items directly
for mapping in result.items:
    print(mapping.name)

# Or iterate (iterates over items)
for mapping in result:
    print(mapping.name)

# Check pagination info
print(f"Showing {len(result)} of {result.total} total")
print(f"Offset: {result.offset}, Limit: {result.limit}")

# Fetch next page
if result.offset + result.limit < result.total:
    next_page = client.mappings.list(offset=result.offset + result.limit,
limit=10)
```

---

8. Data Models

Mapping Models

NodeDefinition

```python
@dataclass
class NodeDefinition:
    """Node table definition for graph mapping."""
    label: str                    # Node label (e.g., "Customer")
    sql: str                      # SQL query to extract data
    primary_key: dict             # {"name": str, "type": str}
    properties: list[dict]        # [{"name": str, "type": str}, ...]

    @classmethod
    def from_dict(cls, data: dict) -> "NodeDefinition": ...

    def to_dict(self) -> dict: ...
```

Example:

```python
node = NodeDefinition(
    label="Customer",
    sql="SELECT id, name, email FROM customers",
    primary_key={"name": "id", "type": "STRING"},
    properties=[
        {"name": "name", "type": "STRING"},
        {"name": "email", "type": "STRING"},
    ],
)
```

EdgeDefinition

```python
@dataclass
class EdgeDefinition:
    """Edge table definition for graph mapping."""
    type: str                     # Edge type (e.g., "PURCHASED")
    from_node: str                # Source node label
    to_node: str                  # Target node label
    sql: str                      # SQL query to extract data
    from_key: str                 # Foreign key to source node
    to_key: str                   # Foreign key to target node
    properties: list[dict]        # [{"name": str, "type": str}, ...]

    @classmethod
    def from_dict(cls, data: dict) -> "EdgeDefinition": ...
```

```
    def to_dict(self) -> dict: ...
```

Example:

```
edge = EdgeDefinition(
    type="PURCHASED",
    from_node="Customer",
    to_node="Product",
    sql="SELECT customer_id, product_id, amount FROM orders",
    from_key="customer_id",
    to_key="product_id",
    properties=[{"name": "amount", "type": "DOUBLE"}],
)
```

MappingVersion

```
@dataclass
class MappingVersion:
    """Immutable mapping version."""
    mapping_id: int
    version: int
    change_description: str | None
    node_definitions: list[NodeDefinition]
    edge_definitions: list[EdgeDefinition]
    created_at: datetime
    created_by: str              # User ID
    created_by_name: str         # Display name

    @classmethod
    def from_dict(cls, data: dict) -> "MappingVersion": ...
```

Mapping

```
@dataclass
class Mapping:
    """Graph mapping definition."""
    id: int
    owner_id: str
    owner_name: str
    name: str
    description: str | None
    current_version: int
    created_at: datetime
```

```
    updated_at: datetime
    ttl: str | None
    inactivity_timeout: str | None
    snapshot_count: int
    version: MappingVersion | None = None  # Included in get() response

    @classmethod
    def from_dict(cls, data: dict) -> "Mapping": ...

    def _repr_html_(self) -> str:
        """Rich HTML display for Jupyter notebooks."""
```

Snapshot Model

```
@dataclass
class Snapshot:
    """Data snapshot from mapping export."""
    id: int
    mapping_id: int
    mapping_name: str
    mapping_version: int
    owner_id: str
    owner_name: str
    name: str
    description: str | None
    gcs_path: str
    size_bytes: int | None
    node_counts: dict[str, int] | None   # {"Customer": 1000, "Product": 500}
    edge_counts: dict[str, int] | None   # {"PURCHASED": 5000}
    status: str                           # pending, creating, ready, failed
    error_message: str | None
    created_at: datetime
    updated_at: datetime
    ttl: str | None
    inactivity_timeout: str | None
    instance_count: int

    @classmethod
    def from_dict(cls, data: dict) -> "Snapshot": ...

    def _repr_html_(self) -> str:
        """Rich HTML display for Jupyter notebooks."""
```

Instance Model

```python
@dataclass
class Instance:
    """Running graph instance."""
    id: int
    snapshot_id: int
    snapshot_name: str
    owner_id: str
    owner_name: str
    name: str
    description: str | None
    instance_url: str | None
    status: str                   # starting, running, stopping, failed
    error_message: str | None
    created_at: datetime
    updated_at: datetime
    started_at: datetime | None
    last_activity_at: datetime | None
    ttl: str | None
    inactivity_timeout: str | None
    memory_usage_bytes: int | None
    disk_usage_bytes: int | None

    @classmethod
    def from_dict(cls, data: dict) -> "Instance": ...

    def _repr_html_(self) -> str:
        """Rich HTML display for Jupyter notebooks."""
```

Schema Model

```python
@dataclass
class Schema:
    """Graph schema with node and relationship tables."""
    nodes: dict[str, dict]          # {label: {primary_key, properties}}
    relationships: dict[str, dict]  # {type: {from, to, properties}}

    @classmethod
    def from_dict(cls, data: dict) -> "Schema": ...

    def _repr_html_(self) -> str:
        """Rich HTML display for Jupyter notebooks."""
```

Algorithm Models

AlgorithmExecution

```python
@dataclass
class AlgorithmExecution:
    """Algorithm execution status and result."""
    execution_id: str
    algorithm: str
    status: str                    # running, completed, failed
    started_at: datetime
    completed_at: datetime | None
    duration_seconds: int | None
    result: dict | None            # Algorithm-specific results
    error: str | None

    @classmethod
    def from_dict(cls, data: dict) -> "AlgorithmExecution": ...
```

Result dict contents vary by algorithm:

```python
# PageRank, centrality algorithms
result = {
    "nodes_updated": 1000,
    "iterations": 15,
    "converged": True,
}

# Connected components
result = {
    "nodes_updated": 1000,
    "component_count": 5,
}

# Shortest path (synchronous)
result = {
    "path": ["A", "B", "C"],
    "length": 2,
    "total_weight": 15.5,  # If weighted
}
```

LockStatus

```python
@dataclass
class LockStatus:
    """Instance lock status."""
    locked: bool
    holder_id: str | None
    holder_name: str | None
    algorithm: str | None
```

```
    acquired_at: datetime | None
    duration_seconds: int | None

    @classmethod
    def from_dict(cls, data: dict) -> "LockStatus": ...
```

Progress Models

SnapshotProgress

```
@dataclass
class SnapshotProgress:
    """Snapshot export progress."""
    id: int
    status: str
    phase: str                      # initializing, exporting_nodes,
exporting_edges, finalizing
    started_at: datetime | None
    completed_at: datetime | None
    duration_seconds: int | None
    steps: list[dict] | None        # [{name, status, started_at, completed_at}]
    completed_steps: int
    total_steps: int

    @classmethod
    def from_dict(cls, data: dict) -> "SnapshotProgress": ...
```

InstanceProgress

```
@dataclass
class InstanceProgress:
    """Instance startup progress."""
    id: int
    status: str
    phase: str                      # scheduling, downloading, loading, ready
    started_at: datetime | None
    ready_at: datetime | None
    startup_duration_seconds: int | None
    steps: list[dict] | None
    completed_steps: int
    total_steps: int

    @classmethod
    def from_dict(cls, data: dict) -> "InstanceProgress": ...
```

Favorite Model

```python
@dataclass
class Favorite:
    """User favorite/bookmark."""
    resource_type: str             # mapping, snapshot, instance
    resource_id: int
    resource_name: str
    resource_owner: str
    created_at: datetime
    resource_exists: bool          # False if resource was deleted

    @classmethod
    def from_dict(cls, data: dict) -> "Favorite": ...
```

---

9. FavoriteResource

Manage user favorites/bookmarks.

Methods

list

```python
def list(self, resource_type: str | None = None) -> list[Favorite]:
    """
    List current user's favorites.

    Args:
        resource_type: Filter by type (mapping, snapshot, instance)

    Returns:
        List of Favorite objects

    Example:
        >>> favorites = client.favorites.list()
        >>> mapping_favorites = client.favorites.list(resource_type="mapping")
    """
```

add

```python
def add(self, resource_type: str, resource_id: int) -> Favorite:
    """
    Add a resource to favorites.

    Args:
        resource_type: Type of resource (mapping, snapshot, instance)
        resource_id: ID of the resource

    Returns:
        Created Favorite object

    Raises:
        ConflictError: If already favorited
        NotFoundError: If resource doesn't exist

    Example:
        >>> client.favorites.add("mapping", 123)
    """
```

remove

```python
def remove(self, resource_type: str, resource_id: int) -> None:
    """
    Remove a resource from favorites.

    Args:
        resource_type: Type of resource
        resource_id: ID of the resource

    Raises:
        NotFoundError: If favorite doesn't exist

    Example:
        >>> client.favorites.remove("mapping", 123)
    """
```

---

10. OpsResource

Operations configuration (requires Ops role).

Methods

get_lifecycle_config

```python
def get_lifecycle_config(self) -> LifecycleConfig:
    """
    Get lifecycle configuration for all resource types.

    Returns:
        LifecycleConfig with mapping, snapshot, and instance settings

    Example:
        >>> config = client.ops.get_lifecycle_config()
        >>> print(config.instance.default_ttl)
    """
```

update_lifecycle_config

```python
def update_lifecycle_config(
    self,
    *,
    mapping: ResourceLifecycleConfig | dict[str, Any] | None = None,
    snapshot: ResourceLifecycleConfig | dict[str, Any] | None = None,
    instance: ResourceLifecycleConfig | dict[str, Any] | None = None,
) -> bool:
    """
    Update lifecycle configuration for resource types.

    Only provided values are updated; omitted values remain unchanged.

    Args:
        mapping: Lifecycle config for mappings
        snapshot: Lifecycle config for snapshots
        instance: Lifecycle config for instances

    Returns:
        True if update succeeded

    Raises:
        ForbiddenError: If user doesn't have Ops role
        ValidationError: If values are invalid

    Example:
        >>> # Update instance default TTL
        >>> client.ops.update_lifecycle_config(
        ...     instance={"default_ttl": "PT24H"}
        ... )
    """
```

get_concurrency_config

```python
def get_concurrency_config(self) -> ConcurrencyConfig:
    """Get instance concurrency limits."""
```

update_concurrency_config

```python
def update_concurrency_config(
    self,
    *,
    per_analyst: int,
    cluster_total: int,
) -> ConcurrencyConfig:
    """
    Update concurrency limits configuration.

    Args:
        per_analyst: Max instances per analyst (1-100)
        cluster_total: Max instances cluster-wide (1-1000)

    Returns:
        Updated ConcurrencyConfig

    Raises:
        ForbiddenError: If user doesn't have Ops role
        ValidationError: If values out of range
    """
```

get_maintenance_mode

```python
def get_maintenance_mode(self) -> MaintenanceMode:
    """Get maintenance mode status."""
```

set_maintenance_mode

```python
def set_maintenance_mode(
    self,
    enabled: bool,
    message: str = "",
) -> MaintenanceMode:
    """Enable or disable maintenance mode."""
```

get_cluster_instances

```python
def get_cluster_instances(self) -> ClusterInstances:
    """
    Get cluster-wide instance summary.

    Returns total instances, breakdowns by status and owner,
    and current capacity limits.

    Returns:
        ClusterInstances with counts and limits

    Raises:
        ForbiddenError: If user doesn't have Ops role
    """
```

get_export_config

```python
def get_export_config(self) -> ExportConfig:
    """
    Get export configuration.

    Returns:
        ExportConfig with max duration settings

    Raises:
        ForbiddenError: If user doesn't have Ops role
    """
```

update_export_config

```python
def update_export_config(
    self,
    *,
    max_duration_seconds: int,
) -> ExportConfig:
    """
    Update export configuration.

    Args:
        max_duration_seconds: Max export job duration (60-86400 seconds)

    Returns:
        Updated ExportConfig

    Raises:
```

```
        ForbiddenError: If user doesn't have Ops role
        ValidationError: If duration out of range
    """
```

get_cluster_health

```
def get_cluster_health(self) -> ClusterHealth:
    """
    Get cluster health status.

    Checks connectivity to database, kubernetes, and starburst.

    Returns:
        ClusterHealth with overall status and component details

    Raises:
        ForbiddenError: If user doesn't have Ops role

    Example:
        >>> health = client.ops.get_cluster_health()
        >>> for name, comp in health.components.items():
        ...     print(f"{name}: {comp.status} ({comp.latency_ms}ms)")
    """
```

get_metrics

```
def get_metrics(self) -> str:
    """
    Get Prometheus metrics from control plane.

    Returns metrics for background jobs, reconciliation loops,
    lifecycle enforcement, and general system health.

    Returns:
        Prometheus metrics in text/plain format

    Raises:
        ForbiddenError: If user doesn't have Ops role

    Example:
        >>> metrics = client.ops.get_metrics()
        >>> assert 'job_name="reconciliation"' in metrics
    """
```

trigger_job

```
def trigger_job(self, job_name: str, reason: str = "manual-trigger") ->
dict[str, Any]:
    """
    Manually trigger background job execution.

    Requires Ops or admin role. Rate limited to 1 trigger per job per minute.

    Args:
        job_name: Job to trigger (reconciliation, lifecycle,
export_reconciliation, schema_cache)
        reason: Reason for manual trigger (audit log)

    Returns:
        Job trigger confirmation with status

    Raises:
        ForbiddenError: If user doesn't have Ops or admin role
        RateLimitError: If job triggered too recently (< 60s)
        NotFoundError: If job name is invalid

    Example:
        >>> client.ops.trigger_job("reconciliation", reason="smoke-test")
        {'job_name': 'reconciliation', 'triggered_at': '...', 'status':
'queued'}
    """
```

get_job_status

```
def get_job_status(self) -> dict[str, Any]:
    """
    Get status of all background jobs.

    Returns:
        Job status information including next run times

    Raises:
        ForbiddenError: If user doesn't have Ops or admin role

    Example:
        >>> status = client.ops.get_job_status()
        >>> for job in status['jobs']:
        ...     print(f"{job['name']}: next run at {job['next_run']}")
    """
```

get_state

```python
def get_state(self) -> dict[str, Any]:
    """
    Get system state summary.

    Returns counts of instances, snapshots, export jobs by status.

    Returns:
        System state with resource counts

    Raises:
        ForbiddenError: If user doesn't have Ops or admin role

    Example:
        >>> state = client.ops.get_state()
        >>> print(f"Instances: {state['instances']['total']}")
        >>> print(f"By status: {state['instances']['by_status']}")
    """
```

get_export_jobs

```python
def get_export_jobs(
    self,
    status: str | None = None,
    limit: int = 100
) -> list[dict[str, Any]]:
    """
    Get export jobs for debugging.

    Args:
        status: Filter by status (pending, claimed, completed, failed)
        limit: Max jobs to return (default 100, max 1000)

    Returns:
        List of export job details

    Raises:
        ForbiddenError: If user doesn't have Ops or admin role
        ValidationError: If status is invalid

    Example:
        >>> claimed = client.ops.get_export_jobs(status="claimed")
        >>> for job in claimed:
        ...     print(f"Job {job['id']} claimed by {job['claimed_by']}")
    """
```

11. HealthResource

Health and status checks.

Methods

check

```python
def check(self) -> HealthStatus:
    """
    Basic health check.

    Returns simple health status without checking dependencies.
    No authentication required.

    Returns:
        HealthStatus with status and version

    Example:
        >>> health = client.health.check()
        >>> print(f"Status: {health.status}, Version: {health.version}")
    """
```

ready

```python
def ready(self) -> HealthStatus:
    """
    Readiness check with database connectivity.

    Checks database connectivity in addition to basic health.
    No authentication required.

    Returns:
        HealthStatus with status, version, and database status

    Example:
        >>> ready = client.health.ready()
        >>> print(f"Status: {ready.status}, DB: {ready.database}")
    """
```

> **Note:** For detailed cluster health with component breakdown, use `client.ops.get_cluster_health()` instead.

12. SchemaResource

Browse Starburst schema metadata.

All operations use cached metadata (refreshed every 24h). Performance is ~5ms per API call (HTTP overhead), ~1µs for cache lookup.

Methods

list_catalogs

```python
def list_catalogs(self) -> list[Catalog]:
    """
    List all cached Starburst catalogs.

    Returns:
        List of Catalog objects (sorted by name)

    Example:
        >>> catalogs = client.schema.list_catalogs()
        >>> for cat in catalogs:
        ...     print(f"{cat.catalog_name}: {cat.schema_count} schemas")
    """
```

list_schemas

```python
def list_schemas(self, catalog: str) -> list[Schema]:
    """
    List all schemas in a catalog.

    Args:
        catalog: Catalog name (e.g., "analytics")

    Returns:
        List of Schema objects

    Raises:
        NotFoundError: Catalog not found in cache

    Example:
        >>> schemas = client.schema.list_schemas("analytics")
        >>> for sch in schemas:
        ...     print(f"{sch.schema_name}: {sch.table_count} tables")
    """
```

list_tables

```python
def list_tables(self, catalog: str, schema: str) -> list[Table]:
    """
    List all tables in a schema.

    Args:
        catalog: Catalog name
        schema: Schema name

    Returns:
        List of Table objects

    Raises:
        NotFoundError: Schema not found in cache

    Example:
        >>> tables = client.schema.list_tables("analytics", "public")
        >>> for tbl in tables:
        ...     print(f"{tbl.table_name} ({tbl.table_type})")
    """
```

list_columns

```python
def list_columns(self, catalog: str, schema: str, table: str) -> list[Column]:
    """
    Get all columns for a table.

    Args:
        catalog: Catalog name
        schema: Schema name
        table: Table name

    Returns:
        List of Column objects (sorted by ordinal_position)

    Raises:
        NotFoundError: Table not found in cache

    Example:
        >>> columns = client.schema.list_columns("analytics", "public",
"users")
        >>> for col in columns:
        ...     print(f"{col.column_name}: {col.data_type}")
    """
```

search_tables

```python
def search_tables(self, pattern: str, limit: int = 100) -> list[Table]:
    """
    Search tables by name pattern (prefix match, case-insensitive).

    Args:
        pattern: Search pattern (e.g., "customer" matches "customers",
"customer_orders")
        limit: Maximum results (default: 100, max: 1000)

    Returns:
        List of Table objects matching pattern

    Example:
        >>> results = client.schema.search_tables("customer", limit=50)
        >>> for tbl in results:
        ...     print(f"{tbl.catalog_name}.{tbl.schema_name}.{tbl.table_name}")
    """
```

search_columns

```python
def search_columns(self, pattern: str, limit: int = 100) -> list[Column]:
    """
    Search columns by name pattern (prefix match, case-insensitive).

    Args:
        pattern: Search pattern (e.g., "email" matches "email",
"email_address")
        limit: Maximum results (default: 100, max: 1000)

    Returns:
        List of Column objects matching pattern

    Example:
        >>> results = client.schema.search_columns("email", limit=50)
        >>> for col in results:
        ...     print(f"{col.catalog_name}.{col.schema_name}.{col.table_name}.
{col.column_name}")
    """
```

admin_refresh

```python
def admin_refresh(self) -> dict:
    """
    Trigger cache refresh (admin only).

    Starts background task to fetch latest metadata from Starburst.
```

```
    Returns immediately.

    Returns:
        Dict with status message

    Raises:
        ForbiddenError: If user doesn't have admin role

    Example:
        >>> result = client.schema.admin_refresh()
        >>> print(result["status"])
    """
```

get_stats

```
def get_stats(self) -> CacheStats:
    """
    Get cache statistics (admin only).

    Returns:
        CacheStats object with counts and metadata

    Raises:
        ForbiddenError: If user doesn't have admin role

    Example:
        >>> stats = client.schema.get_stats()
        >>> print(f"Tables: {stats.total_tables}")
        >>> print(f"Last refresh: {stats.last_refresh}")
    """
```

---

13. AdminResource

Admin-only privileged operations. Requires admin role.

Methods

bulk_delete

```
def bulk_delete(
    self,
    resource_type: str,
```

```
    filters: dict[str, Any],
    reason: str,
    expected_count: int | None = None,
    dry_run: bool = False,
) -> dict[str, Any]:
    """
    Bulk delete resources with safety filters.

    Safety features:
    - At least one filter required
    - Max 100 deletions per request
    - Expected count validation
    - Dry run mode available
    - Full audit logging

    Args:
        resource_type: Resource type (instance, snapshot, mapping)
        filters: Filters to match resources (at least one required):
            - name_prefix: Match resources starting with prefix
            - created_by: Match resources created by username
            - older_than_hours: Match resources older than N hours
            - status: Match resources with specific status
        reason: Reason for deletion (audit log)
        expected_count: Expected number of resources to delete (safety check).
            Must match actual count or operation fails. Get from dry_run first.
        dry_run: If True, return what would be deleted without deleting

    Returns:
        Deletion results with counts and IDs

    Raises:
        ForbiddenError: If user doesn't have Admin role
        ValidationError: If no filters provided, matched > 100, or count
mismatch

    Example:
        >>> # Step 1: Dry run to get count
        >>> result = client.admin.bulk_delete(
        ...     resource_type="instance",
        ...     filters={
        ...         "name_prefix": "E2ETest-",
        ...         "older_than_hours": 24
        ...     },
        ...     reason="cleanup-old-test-instances",
        ...     dry_run=True
        ... )
        >>> print(f"Would delete {result['matched_count']} instances")
        >>>
        >>> # Step 2: Actually delete with expected_count
        >>> result = client.admin.bulk_delete(
        ...     resource_type="instance",
        ...     filters={
        ...         "name_prefix": "E2ETest-",
        ...         "older_than_hours": 24
```

```
        ...     },
        ...     reason="cleanup-old-test-instances",
        ...     expected_count=result['matched_count'],  # Safety check!
        ...     dry_run=False
        ... )
        >>> print(f"Deleted: {result['deleted_count']}")
    """
```

---

14. Exceptions

Exception Hierarchy

```
GraphOLAPError                    # Base exception
    AuthenticationError           # Invalid or missing API key
    PermissionDeniedError         # User lacks permission
        ForbiddenError            # Access forbidden (403)
    NotFoundError                 # Resource not found
    ValidationError               # Request validation failed
    ConflictError                 # Operation conflicts with state
        ResourceLockedError       # Instance locked by algorithm
        ConcurrencyLimitError     # Instance limit exceeded
        DependencyError           # Resource has dependencies
        InvalidStateError         # Invalid for current state
    TimeoutError                  # Operation timed out
        QueryTimeoutError         # Cypher query timeout
        AlgorithmTimeoutError     # Algorithm execution timeout
        SDKTimeoutError           # SDK wait timeout
    RyugraphError                 # Ryugraph/Cypher error
    AlgorithmNotFoundError        # Unknown algorithm
    AlgorithmFailedError          # Algorithm execution failed
    SnapshotFailedError           # Snapshot export failed
    InstanceFailedError           # Instance startup failed
    ServerError                   # Server-side error (5xx)
        ServiceUnavailableError   # Service unavailable (503)
```

Exception Details

ResourceLockedError

```
class ResourceLockedError(ConflictError):
    """Instance is locked by an algorithm."""
```

```python
    @property
    def holder_name(self) -> str | None:
        """Name of the user holding the lock."""

    @property
    def algorithm(self) -> str | None:
        """Algorithm name that acquired the lock."""
```

ConcurrencyLimitError

```python
class ConcurrencyLimitError(ConflictError):
    """Instance creation limit exceeded."""

    @property
    def limit_type(self) -> str | None:
        """Type of limit (per_analyst or cluster_total)."""

    @property
    def current_count(self) -> int | None:
        """Current instance count."""

    @property
    def max_allowed(self) -> int | None:
        """Maximum allowed instances."""
```

---

13. Type Coercion

QueryResult automatically coerces values based on column_types:

Ryugraph Type	Python Type	Notes
STRING	str	UTF-8 text
INT64, INT32, INT16, INT8	int	Integers
DOUBLE, FLOAT	float	Floating point
BOOL	bool	Boolean
DATE	datetime.date	Calendar date

Ryugraph Type	Python Type	Notes
TIMESTAMP	datetime.datetime	Date and time
INTERVAL	datetime.timedelta	Duration
BLOB	bytes	Binary data (base64 decoded)
LIST	list	Recursive coercion
NODE	dict	With _id, _label, properties
REL	dict	With _id, _type, _start, _end, properties
PATH	list	Alternating nodes and rels

Disable coercion to get raw values:

```
result = conn.query("SELECT created_at FROM ...", coerce_types=False)
# result.rows[0][0] = "2024-01-15T10:30:00Z" (string)

result = conn.query("SELECT created_at FROM ...", coerce_types=True)
# result.rows[0][0] = datetime(2024, 1, 15, 10, 30, 0) (datetime)
```

---

14. Complete Example

```
from graph_olap import (
    GraphOLAPClient,
    NodeDefinition,
    EdgeDefinition,
)
from graph_olap_schemas import WrapperType

# Initialize client
client = GraphOLAPClient.from_env()

try:
    # Create a mapping
    mapping = client.mappings.create(
        name="Customer Network",
```

```python
    node_definitions=[
        NodeDefinition(
            label="Customer",
            sql="SELECT id, name, city FROM customers",
            primary_key={"name": "id", "type": "STRING"},
            properties=[
                {"name": "name", "type": "STRING"},
                {"name": "city", "type": "STRING"},
            ],
        ),
    ],
    edge_definitions=[
        EdgeDefinition(
            type="KNOWS",
            from_node="Customer",
            to_node="Customer",
            sql="SELECT from_id, to_id FROM relationships",
            from_key="from_id",
            to_key="to_id",
            properties=[],
        ),
    ],
)

# Create instance directly from mapping (snapshot managed internally)
instance = client.instances.create_from_mapping_and_wait(
    mapping_id=mapping.id,
    name="Analysis Instance",
    wrapper_type=WrapperType.RYUGRAPH,  # Required: RYUGRAPH or FALKORDB
    ttl=24,  # 24 hours (can also use "PT24H")
)
print(f"Instance ready: {instance.status}")

# Connect and query
with client.instances.connect(instance.id) as conn:
    # Basic query
    count = conn.query_scalar("MATCH (n) RETURN count(n)")
    print(f"Total nodes: {count}")

    # Query to DataFrame
    df = conn.query_df("""
        MATCH (c:Customer)
        RETURN c.name, c.city
        ORDER BY c.name
        LIMIT 10
    """)
    print(df)

    # Run PageRank
    exec = conn.algo.pagerank("Customer", "pr_score")
    print(f"PageRank: {exec.result['nodes_updated']} nodes updated")

    # Query top customers
    top = conn.query_df("""
```

```
            MATCH (c:Customer)
            RETURN c.name, c.pr_score
            ORDER BY c.pr_score DESC
            LIMIT 5
        """)
        print("Top customers by PageRank:")
        print(top)

        # Visualize network
        result = conn.query("""
            MATCH (a:Customer)-[r:KNOWS]->(b:Customer)
            RETURN a, r, b
            LIMIT 100
        """)
        result.show()  # Interactive visualization

    # Clean up
    client.instances.terminate(instance.id)

finally:
    client.close()
```

Version History

Version	Date	Changes
1.0.0	TBD	Initial release